

Использование HTTP-клиента

Одним из наиболее распространенных протоколов для организации межсервисного взаимодействия является протокол HTTP(S). Для осуществления запросов по данному протоколу в Wasaby Framework представлен набор типов, при помощи которых можно гибко создавать запросы и принимать ответы. Указанные типы реализованы в библиотеках **sbis-lib** и **sbis-communication** (C++), Python-обертки для них входят в состав модуля sbis (поставляется в составе модуля БЛ "Python Core").

Основными типами, реализующими поддержку клиентской части протокола HTTP(S), являются синхронный и асинхронный HTTP-клиенты. Синхронный HTTP-клиент представлен классом [communication::HTTPClient](#) (C++) и его "оберткой" - [IHTTPClient](#) для использования в Python. Асинхронный HTTP-клиент представлен классом [communication::async_http::Client](#) и его "оберткой" - [AsyncHTTPClient](#) для использования в Python. Далее рассмотрим различные аспекты использования каждого из них более подробно.

В каких случаях использовать HTTP-клиент при разработке на Wasaby Framework?

Но для начала определимся с ответом на вопрос "когда вообще нам требуется использовать какой-либо HTTP-клиент при разработке решений на Wasaby Framework?". Действительно, HTTP-клиент является достаточно низкоуровневым примитивом, старающимся наиболее точно и детально выразить в своем API семантику имплементируемого протокола. Как правило, при взаимодействии между решениями на базе Wasaby Framework используется куда более высокоуровневый API, а именно - **BObject.Invoke**, т.е. RPC-клиент, абстрагирующий от потребителя применяемый низкоуровневый транспорт, а также позволяющий встроить в процесс обработки запроса дополнительные шаги, такие как кэширование и сбор статистики (см. [Синхронный и асинхронный вызов метода БЛ в облаке](#)). Таким образом, у HTTP-клиентов остаются следующие области применения в прикладном коде:

1. Взаимодействие с внешними по отношению к облаку системами (различные сайты госорганов и других компаний).
2. Взаимодействие с компонентами в составе облака не на базе Wasaby Framework, если они предоставляют REST-API и для них нет более высокоуровневой клиентской библиотеки в составе SDK (например, на момент написания данной статьи примером такого компонента является административный API системы управления очередями RabbitMQ).

Отправка запроса и обработка результата

И синхронный, и асинхронный HTTP-клиенты поддерживают работу с основными методами HTTP-запросов: Get, Post, Put, Delete, Head, Copy, Patch и Options.

Для отправки запросов пользователям синхронного HTTP-клиента доступна группа методов класса [HTTPClient](#) для C++ и [IHTTPClient](#) для Python, имена которых соответствуют желаемому HTTP-методу запроса (Get, Post, Put и т.д.). Все без исключения методы из данной группы в качестве первого параметра принимают **url**, на который необходимо сделать запрос. Если соответствующий HTTP-метод предполагает наличие тела запроса, то вторым параметром метода передается содержимое тела. В C++ для представления тела запроса применяется какой-либо наследник класса **std::istream** (при этом должна поддерживаться операция **seek**), а в Python - **str** или **bytes**.

В качестве результата запроса в Python всегда возвращается пара вида (статус ответа типа [HTTPClientStatus](#), полные данные ответа типа **bytes**). Для C++ предусмотрено три способа получения ответа (не для всех HTTP-методов):

- Полный ответ по частям в переданный пользователем поток (наследник **std::ostream**). Данные ответа записываются в поток по мере получения, в случае обрыва соединения в потоке остается неполный ответ. В случае исполнения перенаправления (**redirect**) для потока вызывается метод **seekp(0, std::ios_base::beg)**.
- Полный ответ по частям в переданный пользователем коллбэк типа **std::function< void(char const*, size_t) >**. Данные ответа передаются в коллбэк по мере получения, в случае обрыва соединения коллбэк получает не все данные. В случае исполнения перенаправления (**redirect**) коллбэк никак не оповещается.
- Неполный ответ в виде потока (наследника **std::istream**), "смотрящего" в активное сетевое соединение. Методы с такой сигнатурой возвращают управление вызвавшему в момент получения первой порции тела ответа, не являющегося перенаправлением. По ходу чтения такого ответа могут возникать "затупы" и ошибки из-за необходимости получения данных из сети.

Дополнительно при разработке на C++ можно передать [HttpRequestProgressCallback](#) - специальный коллбэк, периодически вызываемый HTTP-клиентом по ходу исполнения запроса (даже если никакого прогресса по операции нет). С его помощью можно организовать отображение прогресса для пользователя, а также отмену операции, если по каким-то причинам ее результат стал более не нужен.

В следующих примерах показано выполнение Get-запроса.

```
1 # Python
2 # Создаём экземпляр класса HTTP-клиента.
3 http_client = HTTPClient()
4
5 # Устанавливаем предельное время ожидания ответа.
6 http_client.Timeout = 5000
7
8 # Производим Get-запрос.
9 res = http_client.Get( "http://somedomain.com" )
10 if res[0] == HTTPClientStatus.RESPONSE_TIMEOUT:
11     print('Время ожидания ответа от сервера превысило 5 секунд')
12 elif res[0] == HTTPClientStatus.OK:
13     print('Время ожидания ответа от сервера в пределах нормы')
```

Get-запрос на C++ с записью тела ответа в переданный пользователем поток:

```
1 // C++
2 #include <sbis-communication/ihttp_client.hpp>
3 #include <sstream>
4
5 // Создаём HTTP-клиент.
6 communication::HTTPClient http_client;
7
8 // Устанавливаем предельное время ожидания ответа.
9 http_client.SetTimeout( 5000 );
10
11 // Создание потока для записи результата.
12 std::stringstream response;
13 if( http_client.Get( L"http://somedomain.com", response ) ==
    HttpStatus::HTTP_REQUEST_TIMEOUT )
14 {
15
16     // Выводим сообщение об ошибке.
17     std::cout << "timeout elapsed";
18 }
19 else
20 {
21
22     // как-то работаем с response
23 }
```

Get-запрос на C++ с передачей тела ответа в переданный пользователем коллбэк:

```
1 // C++
2 #include <sbis-communication/ihttp_client.hpp>
3 #include <functional>
4
5 // Создаём HTTP-клиент.
6 communication::HTTPClient http_client;
7
8 // Устанавливаем предельное время ожидания ответа.
9 http_client.SetTimeout( 5000 );
10 size_t total_size = 0u;
11 auto const status = http_client.Get( L"http://somedomain.com", \[ &hash
    \]( char const* data, size_t size )
```

```

12 {
13     total_size += size;
14 } );
15 if( status == HttpStatus::HTTP_REQUEST_TIMEOUT )
16 {
17
18     // Выводим сообщение об ошибке.
19     std::cout << "timeout elapsed";
20 }
21 else
22     std::cout << "total bytes received: " << total_size;

```

Get-запрос на C++ с "ранним" возвратом управления вызвавшему:

```

1 // C++
2 #include <sbis-communication/ihttp_client.hpp>
3
4 // Создаём HTTP-клиент.
5 communication::HttpClient http_client;
6
7 // Устанавливаем предельное время ожидания ответа.
8 http_client.SetTimeout( 5000 );
9 auto response = http_client.Get( L"http://somedomain.com" );
10 if( response.mStatus == HttpStatus::HTTP_REQUEST_TIMEOUT )
11 {
12
13     // Выводим сообщение об ошибке.
14     std::cout << "timeout elapsed";
15 }
16 else
17 {
18
19     // как-то работаем с response.mResponse
20 }

```

При использовании асинхронного HTTP-клиента применяется несколько иная схема работы. Т.к. один асинхронный клиент способен исполнять множество запросов параллельно, то все атрибуты и параметры запроса упаковываются в специализированный объект "асинхронный запрос". В итоге общая схема работы выглядит следующим образом:

1. Создаем и настраиваем асинхронный запрос (объект типа [communication::async_http::Request](#) в C++ или [AsyncHTTPRequest](#) в Python).
2. Отправляем запрос нужным HTTP-методом при помощи асинхронного HTTP-клиента (объект типа [communication::async_http::Client](#) в C++ и [AsyncHttpClient](#) в Python), в ответ получаем Future на результат асинхронного запроса (объект типа [communication::async_http::Response](#) в C++ или [AsyncHTTPResponse](#) в Python).
3. Когда нам потребовался результат - ждем готовности Future, забираем результат и работаем с ним.

Рассмотрим на примере исполнения Get-запроса.

```

1 # Python
2 # Создаём асинхронный запрос.
3 request = AsyncHTTPRequest( "http://somedomain.com" )
4 # Создаём асинхронного клиента.
5 client = AsyncHttpClient()
6 # Производим Get-запрос.
7 response_future = client.Get( request )
8 # Занимаемся какими-то своими делами, либо делаем еще запросы
9 ...
10 # Запрашиваем ответ на ранее отправленный запрос
11 response = response_future.get()

```

```

12 # Работаем с ответом response
13 ...

1 // C++
2 #include <sbis-communication/async_http/request.hpp>
3 #include <sbis-communication/async_http/response.hpp>
4 #include <sbis-communication/async_http/client.hpp>
5
6 // Создаём асинхронный запрос.
7 auto request = std::make_unique< communication::async_http::Request >(
    L"http://somedomain.com" );
8
9 // Создаём асинхронного клиента.
10 communication::async_http::Client client;
11
12 // Производим Get-запрос.
13 auto response_future = client.Get( std::move( request ) );
14
15 // Занимаемся какими-то своими делами, либо делаем еще запросы
16 ...
17
18 // Запрашиваем ответ на ранее отправленный запрос
19 auto response = response_future.Get();
20
21 // Работаем с ответом response
22 ...

```

Поддержка HTTP/2

Платформенные HTTP-клиенты полностью поддерживают работу по протоколу HTTP/2. Если удаленный сервер в ходе "переговоров" об установлении защищенного соединения через [ALPN](#) сообщает, что он поддерживает работу по HTTP/2, то клиент пытается установить соединение именно по этому протоколу. Если по каким-то причинам в вашем приложении такое поведение является нежелательным, то использование HTTP/2 может быть отключено путем выставления параметру конфигурации "Ядро.Сервер приложений.ИспользоватьHTTP2" значения "Нет".

Как известно, протокол HTTP/2 является полнодуплексным и позволяет мультиплексировать множество запросов/ответов в одно TCP-соединение, используя так называемые потоки (streams). При высокой пропускной способности сети это позволяет гораздо более эффективно использовать ресурсы и лучше сжимать заголовки при помощи алгоритма **HPACK**. Однако, в случае "плохой" сети с большими задержками и потерей пакетов агрессивное мультиплексирование может приводить к усугублению проблемы [head of line blocking](#). Соответственно, если ваше приложение предусматривает работу в подобных условиях, то может иметь смысл уменьшить максимальное число потоков в одном соединении при помощи конфигурационного параметра "Ядро.Сервер приложений.МаксимальноеЧислоПотоковВСоединенииHTTP2" (по умолчанию его значение равно 5).

Работа с HTTP-заголовками

В рамках типичного запроса мы будем иметь дело с двумя пакетами заголовков: заголовками запроса и заголовками ответа. Для работы с заголовками в коде на C++ применяется класс [HttpHeaders](#), для использования которого необходимо подключить заголовочный файл .

Напомним, что заголовки запроса или ответа по протоколу http представляют собой упорядоченный набор пар вида "имя: значение". Заголовков с одним именем в пакете может быть несколько. Многие из заголовков имеют значение на уровне протокола (например, Content-Length, Transfer-Encoding, Host) и являются необходимыми для организации обмена данными тем или иным образом. Реализация класса [HttpHeaders](#) предполагает, что имена заголовков не содержат не-ascii символов, а значения представимы в кодировке UTF-8. Любые поисковые запросы (например, поиск значения заголовка по имени) предполагают, что регистр символов в имени заголовка не важен. Все данные допущения соответствуют RFC. При использовании настроек по умолчанию имена заголовков переводятся в верхний регистр.

Также следует помнить, что у большинства веб-серверов есть весьма жесткие ограничения на размер одного заголовка и всех заголовков в целом. Поэтому не следует злоупотреблять передачей данных через заголовки, иначе ваш запрос рискует быть отброшенным при прохождении через промежуточный сетевой узел.

Класс [HttpHeaders](#) на момент написания данной статьи не имеет "обертки" в Python. Там для представления пакета заголовков применяется встроенный тип dict (что, естественно, дает чуть меньше гибкости). Однако т.к. в конечном итоге все переданные значения помещаются в [HttpHeaders](#), то к ним также применяются все вышеописанные допущения и правила.

Заголовки запроса

Для задания набора дополнительных заголовков всех последующих запросов при использовании синхронного HTTP-клиента необходимо воспользоваться методом [SetHeaders](#). Для инспекции ранее заданных заголовков можно воспользоваться методом [RequestHeaders](#).

```
1 // C++
2 #include <sbis-lib/types/http_headers.hpp>
3
4 // создаем и заполняем заголовки
5 HttpHeaders headers;
6 headers.Add( L"MyHeaderName"_sv, L"MyHeaderValue"_sv );
7
8 // создаем клиента
9 communication::HTTPClient client;
10
11 // назначаем заголовки клиенту
12 client.SetHeaders( std::move( headers ) );
13
14 // данное условие будет верно
15 assert( client.RequestHeaders().Get( L"MyHeaderName"_sv ) ==
        L"MyHeaderValue"_sv );
```

При разработке на Python для решения аналогичных задач следует воспользоваться свойством [RequestHeaders](#). Следует учитывать, что при чтении данного свойства оно возвращается "по значению", т.е. если вы сохраните полученное значение в переменную и далее попытаете ее как-то изменить, то на HTTPClient-е сделанные изменения никак не отразятся.

```
1 # создаем клиента
2 client = HTTPClient()
3 # задаем заголовки для последующих запросов
4 client.RequestHeaders = { "MyHeaderName" : "MyHeaderValue" }
```

При использовании асинхронного HTTP-клиента пакет дополнительных заголовков для каждого запроса конфигурируется отдельно для каждого объекта [communication::async_http::Request](#) при помощи методов [SetHeaders](#) и [Headers](#). Первый метод позволяет задать весь набор заголовков за один вызов. Второй метод представлен в константном и не-константном вариантах и позволяет либо просто инспектировать ранее заданные заголовки, либо вносить правки в набор "по месту".

```
1 // C++
2 #include <sbis-lib/types/http_headers.hpp>
3 #include <sbis-communication/async_http/request.hpp>
4
5 // создаем и заполняем заголовки
6 HttpHeaders headers;
7 headers.Add( L"MyHeaderName"_sv, L"MyHeaderValue"_sv );
8
9 // создаем запрос
10 auto request = std::make_unique< communication::async_http::Request >
    ();
11
12 // назначаем заголовки клиенту
13 request->SetHeaders( std::move( headers ) );
14
```

```

15 // данное условие будет верно
16 assert( request->Headers().Get( L"MyHeaderName"_sv ) ==
    L"MyHeaderValue"_sv );
17
18 // модифицируем заголовки "по месту"
19 headers->Headers().AddOrReplace( L"MyHeaderName"_sv,
    L"MyNewHeaderValue"_sv );
20
21 // данное условие будет верно
22 assert( request->Headers().Get( L"MyHeaderName"_sv ) ==
    L"MyNewHeaderValue"_sv );

```

При разработке на Python следует пользоваться свойством [Headers](#) класса [AsyncHttpRequest](#), которое по аналогии с синхронным HTTP-клиентом является объектом типа dict. Следует учитывать, что при чтении данного свойства оно возвращается "по значению", т.е. если вы сохраните полученное значение в переменную и далее попытаете ее как-то изменить, то на AsyncHttpRequest-е сделанные изменения никак не отразятся.

```

1 # создаем запрос
2 request = AsyncHttpRequest()
3 # задаем заголовки
4 request.Headers = { "MyHeaderName" : "MyHeaderValue" }

```

Следует отметить, что часть заголовков, необходимых для корректной доставки запроса до исполнителя (например, Content-Length, Transfer-Encoding или Accept-Encoding), HTTP-клиент заполняет самостоятельно и делать это "вручную" не следует (и даже вредно).

Заголовки ответа

Для получения доступа к заголовкам ответа на последний сделанный запрос при использовании синхронного HTTP-клиента следует использовать методы [ResponseHeaders](#) и [TakeResponseHeaders](#). Первый из них дает доступ к полученным заголовкам только на чтение, второй - позволяет забрать пакет заголовков во владение вызывающего кода (через перемещение). Если посредством данного HTTP-клиента запросы не исполнялись или если последний исполненный запрос завершился без получения заголовков ответа (например, сетевой ошибкой), то список заголовков ответа будет пуст. Аналогично, если на объекте [communication::HTTPClient](#) ранее был вызван метод [TakeResponseHeaders](#), то последующие вызовы методов для работы с заголовками ответа будут возвращать пустой пакет.

```

1 // C++
2 #include <sbis-lib/types/http_headers.hpp>
3 #include <sbis-communication/ihttp_client.hpp>
4
5 // создаем клиента
6 communication::HTTPClient client;
7
8 // данное условие будет верно - до исполнения запроса заголовков ответа
  нет
9 assert( !client.ResponseHeaders().IsSet( L"MyHeaderName"_sv ) );
10
11 // ...
12 // делаем запрос, предположим, что он завершился успешно и в составе
  заголовков ответа вернул пару "MyHeaderName: MyHeaderValue"
13 // ...
14 // инспектируем заголовки ответа
15 HttpHeaders const& response_headers = client.ResponseHeaders();
16
17 // данное условие будет верно
18 assert( response_headers.Get( L"MyHeaderName"_sv ) ==
    L"MyHeaderValue"_sv );
19

```

```

20 // забираем заголовки ответа во владение
21 HttpHeaders own_response_headers = client.TakeResponseHeaders();
22
23 // данное условие будет верно - теперь в самом клиенте пакет заголовков
  пуст
24 assert( !client.ResponseHeaders().IsSet( L"MyHeaderName"_sv ) );
25
26 // данное условие будет верно - в полученном нами пакете заголовки
  будут
27 assert( own_response_headers.Get( L"MyHeaderName"_sv ) ==
  L"MyHeaderValue"_sv );

```

При разработке на Python для доступа к заголовкам ответа следует использовать read-only свойство [ResponseHeaders](#) класса [IHttpClient](#).

```

1  # создаем клиента
2  client = IHttpClient()
3  # данное условие будет верно - до исполнения запроса заголовков ответа
  нет
4  assert !client.ResponseHeaders
5  # ...
6  # делаем запрос, предположим, что он завершился успешно и в составе
  заголовков ответа вернул пару "MyHeaderName: MyHeaderValue"
7  # ...
8  # инспектируем заголовки ответа
9  response_headers = client.ResponseHeaders
10 # данное условие будет верно
11 assert response_headers[ "MyHeaderName" ] == "MyHeaderValue"

```

При использовании асинхронного HTTP-клиента доступ к заголовкам ответа осуществляется через объект [communication::async_http::Response](#), соответствующий полученному ответу на запрос. Доступны два метода: [Headers](#) и [TakeHeaders](#). Первый из них дает доступ к полученным заголовкам только на чтение, второй позволяет забрать пакет заголовков во владение вызывающего кода (через перемещение). Если запрос завершился без получения заголовков ответа (например, сетевой ошибкой), то список заголовков ответа будет пуст. Аналогично, если на объекте [communication::async_http::Response](#) ранее был вызван метод [TakeHeaders](#), то последующие вызовы методов для работы с заголовками ответа будут возвращать пустой пакет.

```

1  // C++
2  #include <sbis-lib/types/http_headers.hpp>
3  #include <sbis-communication/async_http/request.hpp>
4  #include <sbis-communication/async_http/response.hpp>
5  #include <sbis-communication/async_http/client.hpp>
6
7  // создаем запрос
8  auto request = std::make_unique< communication::async_http::Request >
  ();
9
10 // ...
11 // настраиваем запрос
12 // ...
13 // создаем асинхронного клиента
14 communication::async_http::Client client;
15
16 // отправляем асинхронный запрос, получаем Future на ответ
17 auto response_future = client.Get( std::move( request ) );
18
19 // ждем готовности Future, получаем сам ответ
20 auto response = response_future.Get();

```

```

21
22 // ...
23 // предположим, что запрос завершился успешно и в составе заголовков
    ответа вернул пару "MyHeaderName: MyHeaderValue"
24 // ...
25 // инспектируем заголовки ответа
26 HttpHeaders const& response_headers = response.Headers();
27
28 // данное условие будет верно
29 assert( response_headers.Get( L"MyHeaderName"_sv ) ==
    L"MyHeaderValue"_sv );
30
31 // забираем заголовки ответа во владение
32 HttpHeaders own_response_headers = response.TakeHeaders();
33
34 // данное условие будет верно - теперь в самом объекте ответа на запрос
    пакет заголовков пуст
35 assert( !response.Headers().IsSet( L"MyHeaderName"_sv ) );
36
37 // данное условие будет верно - в полученном нами пакете заголовки
    будут
38 assert( own_response_headers.Get( L"MyHeaderName"_sv ) ==
    L"MyHeaderValue"_sv );

```

При разработке на Python следует пользоваться read-only свойством [Headers](#) класса [AsyncHTTPResponse](#), которое, по аналогии с синхронным HTTP-клиентом, является объектом типа dict. Следует учитывать, что при чтении данного свойства оно возвращается "по значению", т.е. если вы сохраните полученное значение в переменную и далее попытаете ее как-то изменить, то на AsyncHTTPResponse-е сделанные изменения никак не отразятся.

```

1 # создаем запрос
2 request = AsyncHTTPRequest()
3 # ...
4 # настраиваем запрос
5 # ...
6 # создаем асинхронного клиента
7 client = AsyncHTTPClient()
8 # отправляем асинхронный запрос, получаем Future на ответ
9 response_future = client.Get( request )
10 # ждем готовности Future, получаем сам ответ
11 response = response_future.get()
12 # ...
13 # предположим, что запрос завершился успешно и в составе заголовков
    ответа вернул пару "MyHeaderName: MyHeaderValue"
14 # ...
15 # инспектируем заголовки ответа
16 response_headers = response.Headers;
17 # данное условие будет верно
18 assert response_headers[ "MyHeaderName" ] == "MyHeaderValue"

```

Скрытие значений заголовков с чувствительными данными

На отладочном уровне логирования, а также в случае достаточно долгого исполнения HTTP-запроса платформенные клиенты выводят в логи очень подробную информацию о своей работе. Среди прочего, эта информация содержит все заголовки запроса и все заголовки ответа.

В ряде случаев в этих заголовках могут содержаться чувствительные данные (например, авторизационные токены), попадание которых в логи является крайне нежелательным. Однако при этом полностью отключать логирование HTTP-

запросов не хочется, чтобы не остаться без средств отладки взаимодействия с внешними системами. В этом случае можно воспользоваться специализированными методами `AddPrivateHeader`, которые представлены на уровне классов [communication::HTTPClient](#) (аналог для [Python](#)) и [communication::async_http::Settings](#) (аналог для [Python](#)). Значения всех заголовков, чьи имена были заданы посредством вызова данных методов, не будут попадать в логи. Вместо них будет печататься служебная константа `"\\*\\*\\*value hidden\\*\\*"`.

Работа с cookies

Как известно, протокол HTTP не предполагает установления постоянного соединения с сервером и все запросы являются независимыми друг от друга (stateless). Однако для решения многих практических задач требуется введение понятия сессии для привязки к ней каких-либо данных. Для организации такой сессионности используются cookies.

Как правило, сервер в ответе на какой-либо запрос с клиента дополнительно просит его сохранить некоторые cookies посредством заголовка `Set-Cookie` и ожидает, что все последующие запросы эти назначенные cookies будут содержать в составе заголовка `Cookie`. При этом каждая cookie это не просто пара "имя: значение".

Дополнительно (но возможно неявно), каждая cookie на стороне клиента имеет установленное время жизни, набор доменов, для которых она применима, а также различные дополнительные свойства (`secure`, `http-only`, `same-site`). Работа с cookies, как правило, осуществляется в клиентском ПО конечного пользователя (браузер). В серверных сценариях использование cookies менее типично. Однако в ряде случаев (как правило, это либо обработка перенаправлений, либо многоступенчатые сценарии взаимодействия со сторонними системами, в которых уже сам сервер "эмулирует" клиента) она все-таки требуется. Для предоставления значения cookie используется тип [sbis::Cookie](#), который, по сути, является простым агрегатом для всех свойств одной cookie, кроме имени.

Для хранения значений множества cookies для множества доменов в библиотеке **sbis::communication** применяется класс [CookieStorage](#). На момент написания данной статьи оба обозначенных типа не имеют "обертки" в Python. Вместо них применяется соответствующий API "обертки" HTTP-клиентов. Он предоставляет методы для добавления, удаления (в том числе пакетного, по различным критериям) и инспекции cookies. Способ и семантика его применения несколько отличаются для синхронного и асинхронного клиентов.

Работа с cookies при использовании синхронного HTTP-клиента

При использовании синхронного HTTP-клиента cookies "аккумулируются" в самом объекте [communication::HTTPClient](#) (а на самом деле - в ассоциированном с ним объекте [CookieStorage](#)).

После создания объекта его хранилище cookies пусто, однако по мере исполнения запросов и получения ответов на них оно постепенно наполняется посредством анализа полученных заголовков `Set-Cookie`. Здесь следует особо отметить, что анализируются только и исключительно заголовки ответов. Заголовки `Cookie` у исполняемых клиентом запросов никак не влияют на содержимое хранилища cookies, т.к. на их основе нельзя выполнить анализ дополнительных свойств cookies, что, в свою очередь, является критичным при принятии решения об их применимости в последующих запросах.

Также важно отметить, что, несмотря на то, что физически cookies передаются как значение заголовка `Cookie`, получить к ним доступ через методы для работы с заголовками запроса нельзя (хотя передать дополнительные cookies через свой собственный заголовок `Cookie` можно). Значения cookies синтезируются непосредственно в точке запроса на основе текущего состояния хранилища клиента, целевого url запроса и используемого протокола. В то же время значение заголовка(-ов) ответа `Set-Cookie` по завершении запроса доступно на общих основаниях через соответствующий API.

Для программного доступа к внутреннему хранилищу cookies у [communication::HTTPClient](#) (что требуется **крайне** редко) можно использовать следующие методы:

- [SetCookie](#)
- [GetCookie](#)
- [GetCookieList](#)
- [RemoveCookie](#)
- [ClearCookies](#)
- [ClearSessionCookies](#)

С более подробным описанием каждого из них можно ознакомиться в документации к классу [communication::HTTPClient](#). Рассмотрим вышеописанную механику на примере:

```
1 // C++
2 #include <sbis-lib/types/http_headers.hpp>
3 #include <sbis-communication/ihttp_client.hpp>
4
5 // создаем клиента
```

```

6  communication::HttpClient client;
7  String cookie_value;
8
9  // данное условие будет верно - хранилище кук свеже созданного клиента
   пустое
10 assert( !client.GetCookie( L"somedomain.com", L"trackid", cookie_value
   ) );
11
12 // ...
13 // делаем запрос к somedomain.com/auth, предположим, что он завершился
   успешно и в составе заголовков ответа вернул пару "Set-Cookie:
   trackid=123; expires=Wed, 29-May-2029 00:00:00 GMT"
14 // ...
15 // инспектируем куки - в них мы увидим полученное значение
16 assert( client.GetCookie( L"somedomain.com", L"trackid", cookie_value )
   );
17 assert( cookie_value == L"123"_sv );
18
19 // если мы с помощью объекта client сделаем запрос к
   somedomain.com/data, то в составе заголовков запроса автоматически
   пойдет пара "Cookie: trackid=123"
20 // если же мы с помощью объекта client сделаем запрос к
   someotherdomain.com/data, то в составе заголовков запроса автоматически
   выставленных кук не будет
21 // создаем другого клиента
22 communication::HttpClient other_client;
23
24 // данное условие будет верно - хранилище кук свеже созданного клиента
   пустое, между клиентами они никак не разделяются
25 assert( !other_client.GetCookie( L"somedomain.com", L"trackid",
   cookie_value ) );
26
27 // если мы с помощью объекта other_client сделаем запрос к
   somedomain.com/data, то в составе заголовков запроса автоматически
   выставленных кук не будет

```

При использовании Python-а вышеописанная семантика сохраняется в полном объеме. Для программной манипуляции куками (что требуется **крайне** редко) применяются методы [GetCookie](#) и [SetCookie](#).

```

1  # Python
2  # Создаём экземпляр класса HTTP-клиента.
3  client = IHttpClient()
4  ...
5  # Производим проверку наличия HTTP-cookie
6  if not client.GetCookie("somedomain.com", "trackid"):
7
8      # Если HTTP-cookie не установлены, добавляем их
9      client.SetCookie("somedomain.com", "trackid", "123")

```

Работа с cookies при использовании асинхронного HTTP-клиента

При использовании асинхронного HTTP-клиента семантика работы с cookies существенно отличается от той, что применяется в синхронном клиенте. Причина этого кроется в том, что в случае асинхронного исполнения запросов становится невозможным установить отношение порядка между ними вида "предыдущий-следующий", что, в свою очередь, делает невозможным и детерминированное заполнение некоторого внутреннего хранилища cookies у самого

клиента. По этой причине хранилище cookies ассоциируется с каждым запросом по отдельности (т.е. в составе объекта [communication::async_http::Request](#) присутствует объект [CookieStorage](#), по умолчанию пустой, доступ к которому осуществляется при помощи метода [Cookies](#)), содержимое которого будет применяться при исполнении самого запроса (в том числе при обработке перенаправлений), а полученные в ответе(-ах) cookies будут добавляться в данное хранилище, и в итоге само хранилище будет перемещено в объект ответа на запрос ([communication::async_http::Response](#)). Впоследствии полученные таким образом cookies можно будет назначить на другой запрос. Рассмотрим на примере:

- C++

```
1 // C++
2 #include <sbis-lib/types/http_headers.hpp>
3 #include <sbis-communication/cookie_storage.hpp>
4 #include <sbis-communication/async_http/request.hpp>
5 #include <sbis-communication/async_http/response.hpp>
6 #include <sbis-communication/async_http/client.hpp>
7
8 // создаем запрос
9 auto request = std::make_unique< communication::async_http::Request >
10 ();
11 // ...
12 // настраиваем запрос, кук у нас нет
13 // ...
14 assert( request->Cookies().IsEmpty() );
15
16 // создаем асинхронного клиента
17 communication::async_http::Client client;
18
19 // отправляем асинхронный запрос к somedomain.com/auth, получаем
20 Future на ответ
21 auto response_future = client.Get( std::move( request ) );
22
23 // ждем готовности Future, получаем сам ответ
24 auto response = response_future.Get();
25
26 // ...
27 // предположим, что запрос завершился успешно и в составе заголовков
28 // ответа вернул пару "Set-Cookie: trackid=123; expires=Wed, 29-May-2029
29 // 00:00:00 GMT"
30
31 // ...
32 // инспектируем хранилище кук в ответе - оно будет не пустое
33 assert( !response.Cookies().IsEmpty() );
34
35 // если мы запросим значение конкретной куки, то получим его
36 assert( response.Cookies().GetValue( L"somedomain.com", L"trackid" )
37 == L"123"_sv );
38
39 // создаем следующий запрос
40 auto another_request = std::make_unique<
41 communication::async_http::Request >();
42
43 // ...
44 // настраиваем запрос, кук у нас нет
45 // ...
46 assert( another_request->Cookies().IsEmpty() );
47
48 // однако мы можем передать куки из ответа на предыдущий запрос в
```

```

настраиваемый методом TakeCookies()
43 another_request->Cookies() = response.TakeCookies();
44
45 // также можно использовать код вида another_request->Cookies() =
response.Cookies(), если мы не хотим опустошать хранилище кук в
объекте ответа response
46 assert( !another_request->Cookies().IsEmpty() );
47 assert( another_request->Cookies().GetValue( L"somedomain.com",
L"trackid" ) == L"123"_sv );
48
49 // отправляем асинхронный запрос к somedomain.com/data, получаем
Future на ответ. В составе заголовков запроса будет передана пара
"Cookie: trackid=123"
50 response_future = client.Get( std::move( request ) );

```

- Python

```

1 request = AsyncHTTPRequest( "https://ya.ru" )
2
3 # Ставим куку. Аналогичного эффекта можно добиться, просто проставив
заголовок Cookie
4 request.SetCookie( "cookie_name", "cookie_value" )
5
6 client = AsyncHTTPClient()
7 result = client.Get( request ).get()
8
9 # получаем куку из ответа
10 cookie_value = result.GetCookie( "some_response_cookie_name" )

```

Для Python можно из объекта ответа получить куку по имени с помощью метода [GetCookie](#) и назначить куку на запрос методом [SetCookie](#).

Работа с тайм-аутами

Весьма часто на практике встречаются сценарии, когда ответ на запрос по протоколу HTTP мы готовы ждать лишь в течение определенного периода времени (а не бесконечно), т.е. у запроса имеется тайм-аут. Платформенные HTTP-клиенты предоставляют достаточно гибкие возможности задания тайм-аутов как на весь запрос целиком, так и на отдельные фазы его обработки. В частности, можно сконфигурировать:

- тайм-аут на запрос в целом (Timeout)
- тайм-аут на установление соединения с удаленным сервером (ConnectionTimeout)
- тайм-аут на получение первого байта ответа (StartTransferTimeout)
- тайм-аут на получение следующей порции ответа после предыдущей (ProgressTimeout)

Значения всех тайм-аутов указываются в миллисекундах. Естественно, что между значениями указанных тайм-аутов должны соблюдаться соотношения, соответствующие "вложенности" соответствующих фаз исполнения запроса друг в друга (например, бессмысленно задавать ConnectionTimeout > Timeout, т.к. установление соединения с сервером - это неотъемлемая часть исполнения запроса). По умолчанию ни один из перечисленных тайм-аутов не задан. Точность обслуживания переданных значений тайм-аутов зависит от общего числа исполняемых параллельно HTTP-запросов в системе и загрузки сервера. Как правило, можно рассчитывать на точность в единицы-десятки миллисекунд. Во всех случаях, когда обработка запроса прерывается по тайм-ауту, в качестве статуса ответа на HTTP-запрос возвращается значение HTTP_REQUEST_TIMEOUT (408 Истекло время ожидания от сервера).

При использовании синхронного HTTP-клиента для задания и инспекции тайм-аутов применяются следующие методы:

- [SetTimeout](#) / [Timeout](#)
- [SetStartTransferTimeout](#) / [StartTransferTimeout](#)
- [SetProgressTimeout](#) / [ProgressTimeout](#)
- [SetConnectionTimeout](#) / [ConnectionTimeout](#)

Каждый тайм-аут, настроенный путем соответствующего API-вызова, действует на все последующие запросы, иницируемые данным объектом класса [communication::HTTPClient](#). При передаче значения тайм-аута <= 0 тайм-аут

сбрасывается.

```
1 // C++
2 // Создаём HTTP-клиент.
3 HTTPClient client;
4 ...
5
6 // Проверка установлен ли тайм-аут.
7 if( client.Timeout() == 0 )
8 {
9
10     // Устанавливаем тайм-аут в 1000 мс. на всю операцию
11     client.SetTimeout( 1000 );
12 }
```

При разработке на Python применяются соответствующие свойства класса [IHttpClient](#), например:

```
1 # Python
2 # Создаём экземпляр класса HTTP-клиента.
3 http_client = IHttpClient()
4 ...
5 # Проверка установлен ли тайм-аут.
6 if http_client.Timeout == 0:
7     # Устанавливаем тайм-аут в 1000 мс. на всю операцию
8     http_client.Timeout = 1000
```

При использовании асинхронного HTTP-клиента все тайм-ауты задаются посредством соответствующих методов объекта [communication::async_http::Settings](#) (свойств объекта [AsyncHttpClientSettings](#) в Python) либо на уровне всего клиента, либо на уровне конкретного запроса. Если какие-либо настройки на уровне конкретного запроса выполнялись (т.е. метод [communication::async_http::Request::HasSettings](#) возвращает true), но при этом значения тайм-аутов указаны не были, то "общие" значения тайм-аутов с клиента НЕ применяются.

Работа с перенаправлениями

При выполнении HTTP-запроса на определенный ресурс удаленный сервер может сообщить нам, что по данному url запрашиваемый ресурс более не доступен, однако его можно получить, сделав новый запрос на другой url (возможно, с дополнительными cookies). Это называется перенаправлением (redirect). На уровне протокола для осуществления перенаправления используется комбинация соответствующего статуса ответа (301, 302, 307) и заголовка ответа Location. "Слепое" следование перенаправлениям может создавать проблемы, поэтому в платформенных HTTP-клиентах предусмотрен специализированный API для работы с перенаправлениями. В простейшем случае это разрешение/запрет на автоматическое следование перенаправлениям, в более продвинутом - кастомная политика обработки перенаправлений. По умолчанию все перенаправления разрешены и исполняются автоматически.

При использовании синхронного HTTP-клиента доступен следующий API:

- Метод [DisableAutoRedirect](#) класса [communication::HttpClient](#) в C++ (метод [DisableAutoRedirect](#) класса [IHttpClient](#) в Python) - запрещает автоматическое исполнение перенаправлений. Ответ удаленного сервера возвращается "как есть", т.е. с каким-то из 30X статусов.
- Метод [SetRedirectPolicy](#) класса [communication::HttpClient](#) в C++. Позволяет задать кастомную политику обработки перенаправлений для данного экземпляра HTTP-клиента. На момент написания данной статьи в Python соответствующий API недоступен.

Кастомная политика обработки перенаправлений должна имплементировать интерфейс [sbis::communication::IRedirectPolicy](#) с единственным методом - [CanFollowRedirect](#). Данный метод принимает на вход два url: тот, на который мы пытались сделать запрос и тот, на который сервер нас перенаправляет. Как итог своей работы метод должен вернуть одно из трех значений типа [RedirectReaction](#):

- Follow** - попытаться исполнить перенаправление
- DoNotFollowAndRaiseError** - не исполнять перенаправление и в качестве итога запроса вернуть "ошибочный" статус REDIRECT_CANNOT_BE_FOLLOWED_ERROR
- DoNotFollowAndReturnRequestStatus** - не исполнять перенаправление и завершить обработку запроса "как есть", т.е. с каким-то из 30X статусов

Задание кастомной политики обработки перенаправлений отменяет действие метода `DisableAutoRedirect` и обратно, вызов метода `DisableAutoRedirect` сбрасывает установленную кастомную политику обработки перенаправлений, если таковая была ранее установлена.

При использовании асинхронного HTTP-клиента полностью аналогичный API доступен через класс [communication::async_http::Settings](#) ([AsyncHttpClientSettings](#) в Python).

Также необходимо ограничивать общее число исполняемых перенаправлений в рамках обслуживания одного запроса, иначе можно бесконечно посылать все новые и новые запросы, следуя инструкциям (вероятно, ошибочным или злонамеренным) от удаленного сервера. На момент написания данной статьи в рамках обслуживания одного HTTP-запроса не может быть исполнено более 10 перенаправлений (это значение "зашиито" в исходники платформы). При достижении лимита перенаправлений возвращается статус ответа `REDIRECT_LIMIT_EXCEEDED_ERROR`.

Полный список url-ов, которые были посещены в ходе обслуживания одного запроса (включая все те url, на которые осуществлялись перенаправления) можно получить методом [communication::HttpClient::VisitedUrls](#) ([аналог](#) для Python) при использовании синхронного клиента и [communication::async_http::Response::VisitedUrls](#) ([аналог](#) для Python) при использовании асинхронного.

Статус ответа на HTTP-запрос

Для представления статуса ответа на HTTP-запрос используется перечислимый тип [HttpStatus](#) (или [HttpClientStatus](#) в Python). Все значения данного перечислимого типа можно условно разделить на две большие группы: "настоящие" HTTP-статусы (в том числе "ошибочные", например, 500 Internal Server Error) и "синтетические", которые указывают, что по сути общение по протоколу HTTP по той или иной причине не состоялось (например, мы не смогли соединиться с сервером или не смогли разобрать его ответ). Для классификации полученного значения можно использовать функцию [StatusToHTTPCodeIfValid](#) ([она же](#) для Python) - транслирует значение перечислимого типа `HttpStatus` в пару ("числовой код ответа", "статусная строка, соответствующая данному коду ответа"), если статус не является "синтетическим". В противном случае возвращает `std::nullopt/None`.

Конфигурирование HTTP-клиентов

Конфигурация параметров аутентификации на HTTP-сервере

Для указания параметров доступа к серверу используется классы с одинаковым названием **ServerAuthOptions**: для [Python](#), для [C++](#).

Оба класса содержат следующие параметры конфигурации:

Название параметра	C++	Python	Примечания
Логин	mLogin	Login	Логин аутентификации клиента
Пароль	mPassword	Password	Пароль аутентификации клиента
Флаг игнорирования ошибок безопасности	mIgnoreSecurityErrors	IgnoreSecurityErrors	Определяет работу с недоверенными серверами: подключаться ли к серверу, сертификат которого недоуверен (выдан другому серверу, просрочен или отозван, самоподписан, нельзя проверить)
Сертификат аутентификации клиента	mCertificateOpts	-	Устанавливаются параметры сертификата клиентской аутентификации

Ниже приведены примеры конфигурации:

HTTP-аутентификация

```

2 options = ServerAuthOptions()
3 options.Login = "login"
4 options.Password = "password"
5
6 client.ServerOptions = options

1 // C++
2 communication::ServerAuthOptions options;
3 options.mLogin = L"login"
4 options.mPassword = L"password";
5
6 client->SetServerOptions( std::move( options ) );

```

HTTP-аутентификация по сертификату

```

1 # Python
2 cert_auth_options = sbis.ClientCertificateAuthOptions()
3 cert_auth_options.Certificate = cert_file.Data()
4 # здесь cert_file - объект типа RpcFile с данными сертификата
5 cert_auth_options.Key = cert_key_file.Data()
6 # здесь cert_key_file - объект типа RpcFile с ключом сертификата
7 cert_auth_options.KeyType = "PEM"
8 auth_options = sbis.ServerAuthOptions()
9 auth_options.CertificateOptions = cert_auth_options
10 # здесь client - это объект типа sbis.IHTTPClient
11 client.ServerOptions = auth_options

1 // C++
2 communication::ClientCertificateAuthOptions cert_data;
3
4 // заполняем cert_data
5 ....
6
7 communication::ServerAuthOptions options;
8 options.mCertificateOpts = cert_data;
9 client->SetServerOptions( std::move( options ) );

```

HTTPS, недоверенные серверы

```

1 # Python
2 options = ServerAuthOptions()
3 options.IgnoreSecurityErrors = True
4 client.ServerOptions = options

1 // C++
2 communication::ServerAuthOptions options;
3 options.mIgnoreSecurityErrors = true;
4
5 client->SetServerOptions( std::move( options ) );

```

Взаимодействие с подсистемой балансировки нагрузки

Параметр "ServiceAddress" влияет на систему балансировки загрузки. Это должен быть короткий мнемонический текстовый идентификатор той сетевой службы, с которой мы собираемся осуществить взаимодействие. В параметр можно передать любую строку, например, имя хоста, какую-нибудь мнемонику или неизменяемую часть URL. Главное условие - значение должно описывать службу в целом, а не URL конкретного ресурса на ней и не должно постоянно изменяться.



B ServiceAddress не передаем URL

В противном случае следствием будет утечка памяти и, соответственно, медленная работа сервиса.

```
1 # Python
2 # Устанавливаем имя вызываемого сервиса для системы балансировки. При
  взаимодействии с внешним сервисом установите имя, характеризующее
  вызываемый сервис.
3 cl.SetServiceAddress( 'cloudstore' )
4
5 # Выполняем запрос.
6 reply = cl.Get( 'https://dev-cloudstore-
  proxy1.unix.tensor.ru:8080/v1/AUTH_online/doc31887/fefd1b6d-ebe2-4c07-
  b681-48cacae31887' )
```

Конфигурация настроек прокси-сервера

Если доступ к ресурсу осуществляется через прокси-сервер, то в конфигурации HTTP-клиента нужно установить его настройки:

1. **Адрес прокси-сервера.** Его устанавливают в формате "адрес:порт".
2. **Настройки авторизации.** Если для доступа к прокси-серверу требуется пройти авторизацию, то в настройках указывают логин и пароль пользователя.

Для конфигурации настроек в Python используют класс [InternetProxy](#) и атрибут [Proxy](#).

```
1 # Python
2 # Создаём экземпляр класса для настройки прокси-сервера.
3 proxy_config = InternetProxy()
4
5 # Устанавливаем адрес прокси-сервера.
6 proxy_config.Url = 'corp-iasc.corp.tensor.ru:8080'
7
8 # Устанавливаем логин для авторизации на прокси-сервере.
9 proxy_config.Login = someString1
10
11 # Устанавливаем пароль для авторизации на прокси-сервере.
12 proxy_config.Password = someString2
13 ...
14 # Создаём экземпляр класса HTTP-клиента.
15 http_client = HTTPClient()
16
17 # Устанавливаем конфигурацию прокси-сервера для HTTP-клиента.
18 http_client.Proxy = proxy_config
```

Для конфигурации настроек в C++ используют структуру [InternetProxy](#) и метод [Proxy](#).

```
1 // C++
2 // Создаём экземпляр класса структуры для настройки прокси-сервера.
3 communication::InternetProxy& p1;
4
5 // Устанавливаем адрес прокси-сервера.
6 p1.mLogin = L"corp-iasc.corp.tensor.ru:8080";
7
8 // Устанавливаем логин для авторизации на прокси-сервере.
9 p1.mLogin = someString1;
10
11 // Устанавливаем пароль для авторизации на прокси-сервере.
12 p1.mPassword = someString1;
```



```

13 ...
14 // Создаём HTTP-клиент.
15 std::unique_ptr<communication::IHTTPClient> http_client(
    communication::CreateHTTPClient() )
16
17 // Устанавливаем конфигурацию прокси-сервера для HTTP-клиента.
18 http_client->setProxy( p1 );
19

```

Обратная задача — получить уже установленные настройки.

Для получения конфигурации прокси-сервера в Python используют класс [InternetProxy](#) и атрибут [Proxy](#).

```

1 # Python
2 # Ранее создан экземпляр класса HTTP-клиента в переменной http_client.
3 # Создаём экземпляр класса для настройки прокси-сервера.
4 proxy_config = InternetProxy()
5
6 # Получаем конфигурацию.
7 proxy_config = http_client.Proxy

```

Для получения конфигурации прокси-сервера в C++ используют метод [Proxy](#).

```

1 // C++
2 // Ранее объявлен HTTP-клиент в переменной http_client.
3 // Возвращаем в переменную p1 настройки прокси-сервера.
4 const communication::InternetProxy p1 = http_client->Proxy();

```

Конфигурация доменной аутентификации

Поддерживается работа доменной аутентификации на сервере-хосте.

Для настройки доменной аутентификации в Python используют атрибуты [AutomaticAuth](#).

```

1 # Python
2 # Создаём экземпляр класса HTTP-клиента.
3 http_client = IHTTPClient()
4 ...
5 # Проверка: установлен ли параметр.
6 if not http_client.AutomaticAuth:
7
8     # Разрешаем доменную аутентификацию.
9     http_client.AutomaticAuth = True

```

Для настройки доменной аутентификации в C++ используют методы [SetAutomaticAuth](#) и [AutomaticAuth](#).

```

1 // C++
2 // Создаём HTTP-клиент.
3 std::unique_ptr<communication::IHTTPClient> http_client(
    communication::CreateHTTPClient() )
4 ...
5 // Проверка: установлен ли параметр.
6 if( !http_client->AutomaticAuth() )
7 {
8     // Разрешаем доменную аутентификацию.
9     http_client->SetAutomaticAuth( true );
10 }

```

Ограничения на использование сторонних HTTP-клиентов

Для клиент-серверного взаимодействия в исходном коде на Python запрещено использовать "urllib.request", "http.client" и любые другие неплатформенные библиотеки. Использование неплатформенного http-клиента это:

- при использовании корутин - отсутствие кооперативного переключения контекста при сетевых вызовах, и, как следствие, "зависание" рабочего процесса сервиса до окончания обработки запроса
- отсутствие отслеживания внешних вызовов, как следствие — [сервис затороустойчивости](#) не может помочь
- отсутствие интегрированных в экосистему облака средств логирования и диагностики

Далее приведем примеры перехода на платформенный класс и описание ошибок, возникающих с использованием нереконмендуемых библиотек.

Пример 1. Пример перехода от библиотек "urllib.request" и "http.client" на платформенный класс для работы с HTTP-клиентом.

- Библиотека "urllib.request".

```
1 # Импортируем библиотеку.
2 from urllib.request import urlopen
3
4 # Производим запрос.
5 response = urlopen( 'https://online.sbis.ru/auth/service/' ).read()
6
7 # Выводим результат.
8 print( response.read() )
```

- Библиотека "http.client".

```
1 # Импортируем библиотеку.
2 import http.client
3
4 # Создаём HTTPS-соединение с online.sbis.ru.
5 connection = http.client.HTTPSConnection( 'online.sbis.ru' )
6
7 # Указываем адрес страницы, которую хотим запросить.
8 connection.request( '/auth/service/' )
9
10 # Выполняем запрос.
11 response = connection.getresponse()
12
13 # Выводим результат.
14 print( response.read() )
```

- Платформенный класс "IHTTPClient".

```
1 # Создаём экземпляр класса HTTP-клиента.
2 connection = IHTTPClient()
3
4 # Производим Get-запрос по адресу.
5 response = connection.Get( 'https://online.sbis.ru/auth/service/' )
6
7 # Декодируем ответ от сервера.
8 result = json.loads( response[1].decode() )
9
10 # Выводим результат.
11 print( result )
```

Пример 2. Этот пример демонстрирует возникновение ошибки при использовании библиотеки "http.client". Она была найдена на dev-online, где не работала регистрация через VK и Facebook. Ошибочный код находился в методе "САП.AuthenticateByOpenAuth". Источник: [ссылка](#).

- Исходный код, который приводил к ошибке.

```

1 # Подключена библиотека http.client.
2 from http import client
3 ...
4 def ConnectResponse (self, url, quer):
5
6     # Открываем соединение для работы с запросами.
7     conn = client.HTTPSConnection(url)
8
9     # Производим Get-запрос по адресу.
10    conn.request("GET", quer )
11
12    # Декодируем ответ от сервера.
13    result = json.loads( conn.getresponse().read().decode() )
14
15    # Закрываем соединение.
16    conn.close()
17    return result

```

- Ошибка была устранена использованием класса `HTTPClient`. Исходный код стал компактнее.

```

1 # Подключать библиотеку http.client уже не нужно, только sbis
2 ...
3 def ConnectResponse (self, url, quer):
4
5     # Создаём экземпляр класса для работы запросами.
6     conn = HTTPClient()
7
8     # Производим Get-запрос по адресу.
9     resp = conn.Get( "https://" + url + quer )
10
11    # Декодируем ответ от сервера.
12    result = json.loads( resp[1].decode() )
13    return result

```